

GLMFuzz: Vulnerability Knowledge Guided Prompting for Efficient Network Protocol Fuzzing

Shengkai Zhu¹, Shuitao Gan^{2,*}, Xiaofeng Wang^{3,*}, and Yan Nan²

¹School of Artificial Intelligence and Computer Science, Jiangnan University, China

²Laboratory for Advanced Computing and Intelligence Engineering, Wuxi, China

³Pengcheng Laboratory, Shenzhen, China

6233111078@stu.jiangnan.edu.cn, ganshuitao@gmail.com, wangxf@pcl.ac.cn, n_yan11111@163.com

*corresponding author

Abstract—Security vulnerabilities in network protocol implementations have increased rapidly, posing serious threats to network infrastructure. Fuzzing has emerged as a primary method for detecting these vulnerabilities. However, due to the stateful nature of protocol implementations, existing mutation-based protocol fuzzing methods face challenges in generating high-quality initial seeds and developing effective mutation strategies. In this paper, we propose GLMFuzz, which is a vulnerability-knowledge-guided fuzzing method for protocol implementations based on large language models (LLMs). By analyzing historical vulnerability sequences, which contain special states and field information that can trigger boundary conditions and exceptional behaviors in protocol implementations, GLMFuzz constructs a vulnerability knowledge vector database and utilizes LLM to generate diverse initial seeds. During the fuzzing process, GLMFuzz uses vulnerability sequences to guide the LLM in performing directed mutations on the message, producing test cases that cover unusual behaviors. In addition, we design an automatic prompt engineering algorithm to dynamically adjust prompt templates based on fuzzing results. To validate the performance of GLMFuzz, we conduct a comparative evaluation with two state-of-the-art (SOTA) tools, AFLNET and CHATAFL. The experimental results demonstrate that GLMFuzz achieves average improvements of 54.18% and 16.48% in state transitions, 14.57% and 4.97% in state coverage, 10.61% and 7.01% in code coverage compared to AFLNET and CHATAFL, respectively, and identifies a previously unknown vulnerability in protocol implementations, which has been assigned CVE identifier.

Keywords—fuzzing; protocol implementations; large language models; automatic prompt engineering

1. INTRODUCTION

Network protocol implementations are essential when exchanging information between network infrastructures, as they not only define the rules for data transmission, but also ensure its efficiency and accuracy [1]. However, owing to the necessity for protocol implementations to expose interfaces on the network [2], they often suffer from malicious traffic attacks, leading to further attacks on network infrastructure and causing serious security issues [3][4]. According to recent statistics [5][6], software vulnerabilities in protocol imple-

mentations have been increasing. Therefore, detecting and addressing these vulnerabilities becomes a critical priority.

Fuzzing, which identifies potential vulnerabilities by generating numerous test cases as inputs and observing the behavior of program under test (PUT) for anomalies [7], has become a primary technique for vulnerability discovery in protocol implementations [8][9]. Unlike other PUTs, the stateful nature of protocol implementations poses unique challenges to fuzzing [10]. Specifically, protocol implementations operate as state transition systems based on finite-state machines [11], where the systems trigger only corresponding state transitions and execute processing procedures when the received message sequences strictly adhere to a protocol-specified order [12].

Mutation-based protocol fuzzing methods [13][14][15] capture messages from a single interaction between a client and a server and sequentially combine all the messages sent by the client into an initial seed as test inputs. During the fuzzing process, they select request sequences that can reach specific states and perform simple mutations on the message fields to generate test cases. Consequently, the performance of mutation-based protocol fuzzing methods is constrained by the quality of the initial seeds, and simple mutation strategies cannot cover a deep state space. CHATAFL [12] leverages an LLM to add request messages for the initial seed corpus and uses historical interaction messages to guide the LLM in generating potential request sequences to surpass coverage plateau. However, the LLM tends to generate standardized messages that comply with protocol specifications and fails to produce unusual messages that cover specific states and boundary conditions in protocol implementations, thus leading to limited fuzzing effectiveness. Furthermore, CHATAFL employs static prompt templates for LLM interactions and is unable to dynamically optimize template structures based on fuzzing results, thereby restricting the ability of the LLM to generate high-potential test cases.

In this paper, we propose GLMFuzz, a vulnerability-knowledge-guided fuzzing method for protocol implementations based on LLMs. Historical vulnerability sequences frequently contain special state and field information that can trigger boundary conditions and exceptional behaviors in protocol implementations [16], which are crucial for effective protocol fuzzing. Therefore, we collect these sequence fragments and extract sequence features to construct a vulnerabil-

ity knowledge vector database. GLMFuzz employs Retrieval-Augmented Generation (RAG) [17] to retrieve vulnerability knowledge and leverages LLM to generate message sequences with similar vulnerability features that also comply with protocol specifications, thereby increasing the diversity of request messages in the initial seed corpus. During the fuzzing process, GLMFuzz combines these vulnerability sequences with different standard sequences, guiding LLM to perform directed mutations on the seeds and thus generating test cases that can cover more interesting paths. In addition, we also propose a scheduling strategy to choose the vulnerability sequences with the highest reward values.

Furthermore, during the process of interacting with LLM through prompt engineering, the quality of the template prompts significantly impacts the model's output, where well-crafted templates can precisely guide LLM to generate high-value feedback. Therefore, GLMFuzz develops an automatic prompt engineering algorithm to evaluate the quality of prompt templates based on fuzzing results and flexibly employs three optimization strategies to adjust the prompt templates, thereby guiding LLM to generate more diverse inputs.

We conduct experiments on the protocol fuzzer benchmark PROFUZZBENCH [18] and compare them with the two current SOTA fuzzers, CHATAFL [12] and AFLNET [13]. The experimental results demonstrate that, compared with AFLNET and CHATAFL, GLMFuzz achieves average improvements in state transitions, state coverage, and code coverage. In addition to enhanced coverage, GLMFuzz also discovers a previously unknown vulnerability in protocol implementations.

In summary, the contributions of this paper are as follows:

- We propose GLMFuzz, a vulnerability-knowledge-guided fuzzing method for protocol implementations based on LLMs, which uses special states and field knowledge in historical vulnerability sequences to increase the quality of initial seeds and guide LLM in performing directed mutations to seeds.
- We propose an automatic prompt engineering algorithm that dynamically adjusts prompt templates via three optimization strategies by evaluating fuzzing results, guiding the LLM to generate more targeted inputs.
- We conduct an evaluation of GLMFuzz. The results show that GLMFuzz outperforms the current SOTA fuzzing tools, CHATAFL and AFLNET, in terms of coverage of the state space and code space. We also identify a previously unknown vulnerability.

2. BACKGROUND AND MOTIVATION

2.1 Protocol Fuzzing

Fuzzing is an effective method to discover vulnerabilities among diverse software [19][20][21], and has been widely adopted in network protocol implementations. The main approach of protocol fuzzing is to treat the fuzzing tool as the client side, continuously generating messages that satisfy the protocol syntax to interact with the server under test (SUT)

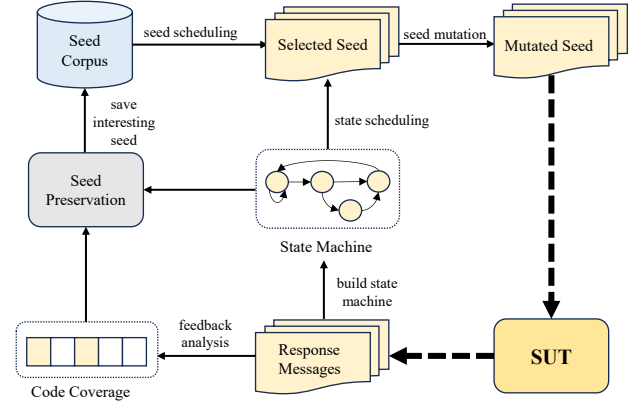


Figure 1. The general workflow of mutation-based protocol fuzzing.

[22]. The protocol fuzzing can be divided into generation-based fuzzing and mutation-based fuzzing depending on how the protocol fuzzing tool generates message sequences.

Generation-based protocol fuzzing requires users to provide configurations that strictly follow the specifications of the target protocol. Peach [23], for instance, requires users to write Pit configuration files via the XML language, filling in corresponding state modules and data modules based on protocol states and syntax. SNOOZE [24] offers users a set of fuzzing primitives, enabling testers to extract protocol specifications from standard documents and fill in message objects, message fields, and so on. However, owing to the requirement for a deep understanding of the protocol implementation in generation-based fuzzing, this approach often fails to fully cover the deep logic of the protocol and is susceptible to errors in field generation. As a result, current research has increasingly focused on mutation-based fuzzing.

Mutation-based protocol fuzzing eliminates the need for manual protocol configurations input. As shown in Figure 1, these methods mutate pre-stored message sequences as inputs and utilize feedback responses from the server to update the state machine. AFLNET [13] aggregates client-server exchange messages into unified seeds and updates the finite-state machine through response message analysis. StateAFL [14] uses snapshot technology to track server-side memory allocation as the unique service state. NSFUZZ [15] automatically models server states using code-level state variables and builds a state transition tree based on these variables. During the fuzzing process, these fuzzing tools select the target state, choose corresponding seeds from the initial seed corpus, and continuously generate test cases via byte-level or region-level mutations.

However, mutation-based protocol fuzzing still faces several limitations. The initial seed corpus lacks quality assurance, and pre-stored messages cannot effectively probe hidden protocol states. Additionally, mutation strategies are inefficient. Random byte mutations compromise the structural integrity of protocol messages, which prevents the server from processing

them correctly.

2.2 Large Language Models

Large language models (LLMs) have shown remarkable abilities in natural language processing, particularly in text generating [25][26] and summarizing [27][28]. To overcome the limitations of traditional fuzzing methods in seed generation and mutation, researchers are exploring the integration of LLMs with fuzzing techniques for test case generation [29] and vulnerability detection [30].

For example, TitanFuzz [31] combines LLM with deep learning library fuzzing, which uses CodeX to generate substantial amounts of deep learning API code while using heuristic algorithms to optimize the fuzzing process. Similarly, Fuzz4All [32] employs GPT-4 as the input generation and mutation engine, allowing fuzzing on various programs, compilers, and API libraries, thus broadening the applicability of fuzzing techniques. ChatFuzz [33] introduces an innovative fuzzing framework that merges LLM with gray-box fuzzing. In this approach, the mutation strategy involves seed selection and generation of mutated seeds using ChatGPT. In the realm of protocol fuzzing, ChatAFL [12] uses LLM to abstract network protocol syntax and generate messages to effectively expand the initial seed corpus.

2.3 Motivation

The effectiveness of protocol fuzzing is constrained primarily by the quality of the initial seeds and mutation strategies. CHATAFL improves seed diversity by adding protocol messages to the initial seed corpus and uses LLM to generate subsequent request sequences based on historical interaction context, thereby surpassing the coverage plateau during the mutation stage. However, LLM-generated inputs are primarily ordinary messages and cannot cover edge cases in the protocol implementations, which limits improvements in seed quality and mutation strategy efficacy.

Zhao et al.[16] revealed that historical vulnerability programs contain core code segments capable of covering boundary conditions. Combining these segments with standard seeds can generate more effective test cases. For protocol implementations, historical vulnerability sequences inherently contain abnormal features of protocol states and field information, which can provide critical guidance for LLM to generate high-quality test cases.

Figure 2 illustrates the message sequences that trigger the CVE-2021-39283 vulnerability. These messages consist of a series of repeated “SETUP” and “PLAY” requests, through which the client can cause an assertion failure in the server’s program, leading to a service crash. Due to the lack of such vulnerability sequence knowledge, CHATAFL struggles to generate high-quality inputs. Therefore, we design a vulnerability-knowledge-guided method that uses vulnerability sequences to guide the LLM in producing messages that cover boundary conditions or exceptional behaviors to increase the diversity of initial seeds and improve mutation efficiency.

```
SETUP rtsp://127.0.0.1:8554/mpeg1or2AudioVideoTest/track2 RTSP/1.0\r\n
CSeq: << CSeq>>\r\n
Transport: RTP/AVP;unicast;client_port=45546-45547\r\n
Session: <<SessionID>>\r\n
\r\n
PLAY rtsp://127.0.0.1:8554/mpeg1or2AudioVideoTest/ RTSP/1.0\r\n
CSeq: << CSeq>>\r\n
Session: <<SessionID>>\r\n
Range: npt=0.000-\r\n
\r\n
SETUP rtsp://127.0.0.1:8554/mpeg1or2AudioVideoTest/track1 RTSP/1.0\r\n
CSeq: << CSeq>>\r\n
Transport: RTP/AVP;unicast;client_port=50778.08.28)\r\n
Session: <<SessionID>>\r\n
\r\n
SETUP rtsp://127.0.0.1:8554/mpeg1or2AudioVideoTest/track2 RTSP/1.0\r\n
CSeq: << CSeq>>\r\n
Transport: RTP/AVP;unicast;client_port=45546-45547\r\n
Session: <<SessionID>>\r\n
\r\n
PLAY rtsp://127.0.0.1:8554/mpeg1or2AudioVideoTest/ RTSP/1.0\r\n
CSeq: << CSeq>>\r\n
Session: <<SessionID>>\r\n
Range: npt=0.000-\r\n
\r\n
```

Figure 2. Example of bug sequences that trigger CVE-2021-39283.

Furthermore, the static prompt templates used in CHATAFL fail to adapt mutation strategies based on real-time metrics, such as code coverage. This limitation results in the repetitive generation of ineffective test cases, especially during the middle and later stages of fuzzing. To solve this problem, we propose an automatic prompt engineering algorithm to adapt the prompt template based on fuzzing results. When ineffective test cases are detected, the algorithm refines the prompt template with three optimization strategies.

3. DESIGN

Figure 3 illustrates the structure of our approach, GLMFuzz, which consists of four main components: database construction, seeds expansion, fuzzing loop, and automatic prompt engineering.

3.1 Database Construction

As discussed in Section 2.3, historical vulnerability sequences often contain specific protocol state and field information. To obtain these sequences, we gather data primarily from two sources: (1) the CVE database and official patch logs, which typically publish standardized bug reports containing the message sequences necessary for reproducing bugs. (2) User-submitted issues on GitHub, where users often provide detailed descriptions of problems along with relevant message sequences.

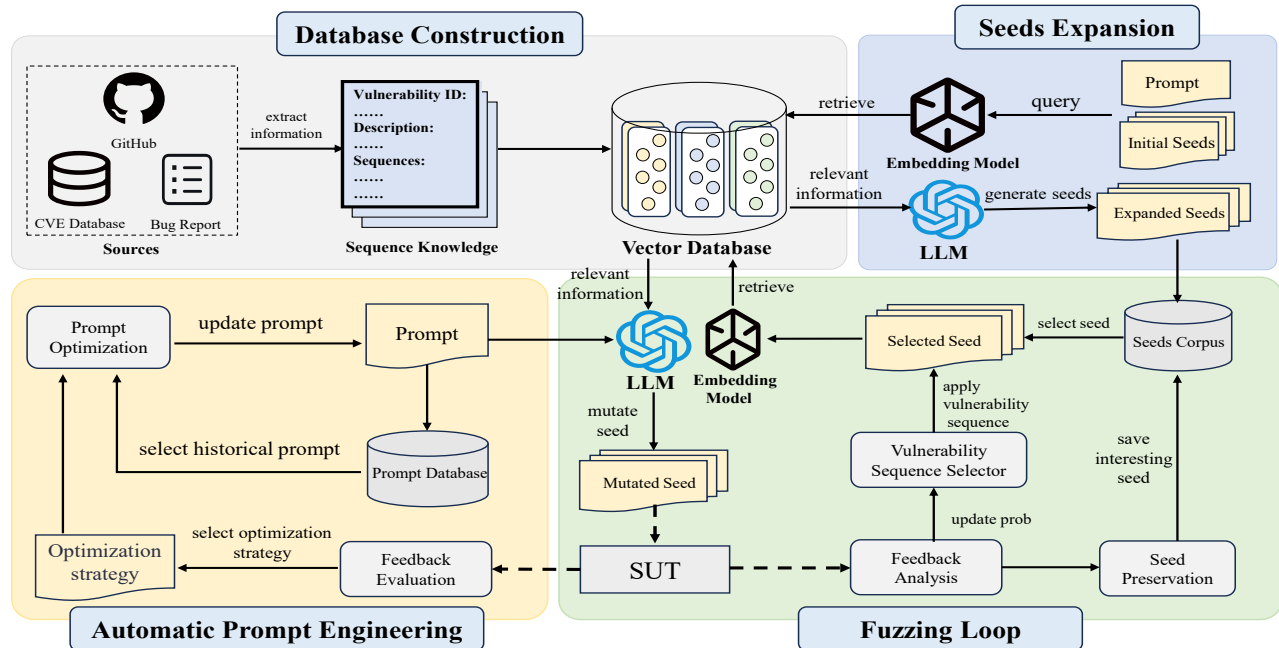


Figure 3. Overview of GLMFuzz.

We extract related vulnerability information from these platforms and organize it by vulnerability ID, description, and sequences to form a knowledge document. We assign vulnerability IDs based on CVE identifiers and issue titles. The description field includes vulnerability sequence features, such as repeated “SETUP” and “PLAY” requests to trigger CVE-2021-39283. For protocol sequences, we embed sequences with type-constrained placeholders, allowing the LLM to dynamically generate test cases with targeted mutation values based on the context.

However, during the content generation process, LLM often produces information that does not exist, which is a phenomenon known as the “hallucination” problem [34]. To reduce the occurrence of hallucination issues, GLMFuzz employs RAG [17] to generate relevant information. Specifically, we construct a comprehensive knowledge base on collected sequence information and employ embedding models to extract semantic feature vectors from the information, thereby establishing a vector database [35]. When GLMFuzz selects a vulnerability sequence knowledge to guide LLM in generating test cases, RAG first retrieves the most semantically relevant context from the vector database via an embedding model. Then it combines this retrieved information with the user prompt template to create an enhanced prompt, which is finally used by LLM to generate a precise output.

3.2 Seeds Expansion

During the initialization stage, we improve the quality of the initial seeds by incorporating vulnerability knowledge. To ensure that GLMFuzz generates stable initial seeds based

on selected vulnerability sequences, we employed prompt engineering and few-shot learning methods [31].

The specific prompt templates are illustrated in Figure 4. First, GLMFuzz fills the vulnerability identifier into the first template and uses the LLM to retrieve the sequence knowledge from the vector database, thereby generating a sequence template containing vulnerability features. Then, GLMFuzz randomly selects the initial seed message and inserts it with the generated sequence template into the corresponding positions of the second template, forming a structured request input for the LLM. Based on the contextual information of the initial seed message, the LLM parses the semantic content and dynamically fills in the field information of the vulnerability sequence template, and finally outputs expanded messages that comply with the protocol syntax and carry targeted vulnerability features. Subsequently, GLMFuzz normalizes the messages generated by the LLM, which involves removing irrelevant characters, converting escape characters, and verifying the protocol format.

Finally, GLMFuzz appends the normalized protocol messages to the initial message sequences, thereby forming a new initial seed, which is then stored in the initial seed corpus for fuzzing.

3.3 Fuzzing Loop

3.3.1 Fuzzing with Directed Mutation

As fuzzing progresses, traditional byte-level mutation can no longer generate message sequences that comprehensively cover the deep state space of protocols. GLMFuzz employs a directed mutation based on vulnerability sequences, leveraging

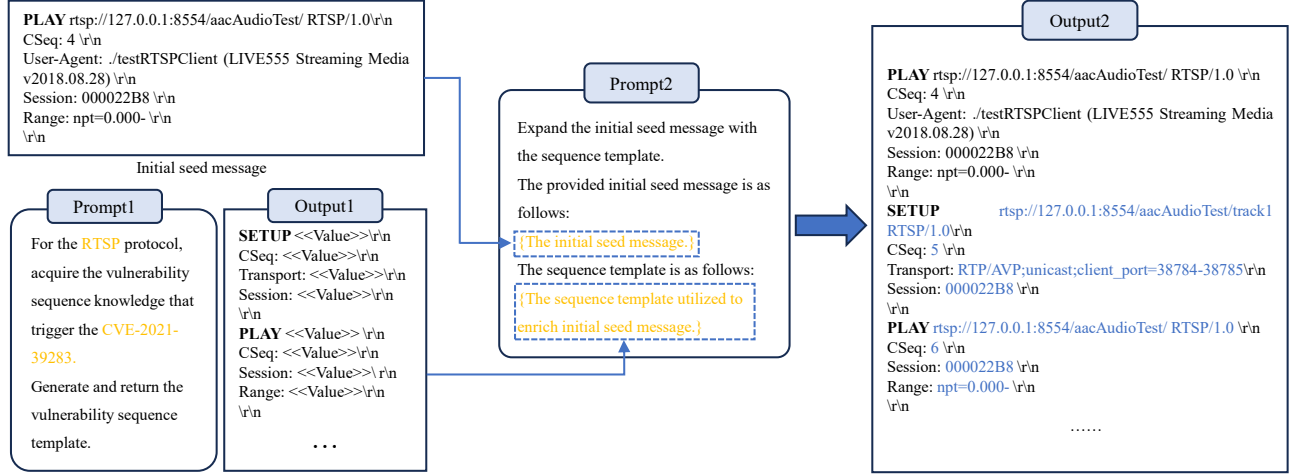


Figure 4. Examples of the prompts and model outputs for enriching the initial seed.

them to mutate the test seeds and explore the state space of the protocol.

GLMFuzz integrates existing vulnerability sequences and test sequences to trigger interesting interactions between them. Specifically, in state S , GLMFuzz selects the sequences $\langle M_1, M_2, M_3 \rangle$, where M_1 is the prerequisite messages required to reach state S , M_2 is the message that needs to be mutated in the desired state, and M_3 is the subsequent message. Then it randomly selects the vulnerability sequences M_h generated in the initialization stage and embeds them into the mutated seed sequences, forming new mutated sequences $\langle M_1, M_2, M_h, M_3 \rangle$, which are sequentially sent to the server for fuzzing. Moreover, to simulate the instability of the network environment, GLMFuzz also randomly shuffles the mutated sequence or duplicate parts of the messages.

When the number of invalid test cases reaches a predefined threshold, GLMFuzz uses vulnerability features to guide LLM in performing directed mutation on seeds. GLMFuzz fills the target vulnerability identifier in the prompt template, similar to the first prompt template shown in Figure 4, to obtain the sequence template. Then, as illustrated in Figure 5, GLMFuzz embeds the target message M_2 and the generated vulnerability sequence template into the prompt template, and utilizes the LLM to generate the mutated sequence M'_2 . To address the instability of LLM-based mutation, constraint rules are embedded in the prompt template. These rules ensure that the sequences generated by LLM must cover the features of the vulnerability sequences, strictly comply with protocol format specifications, and ensure the logical correctness of critical fields such as session identifiers. After satisfying these constraints, the rules further guide the LLM to execute mutation strategies, including injecting out-of-range numbers, inserting over-sized string payloads into protocol fields, and so on. Ultimately, GLMFuzz combines the returned sequence M'_2 with M_1 and M_3 to form new sequences $\langle M_1, M'_2, M_3 \rangle$ and sends them to the server for fuzzing.

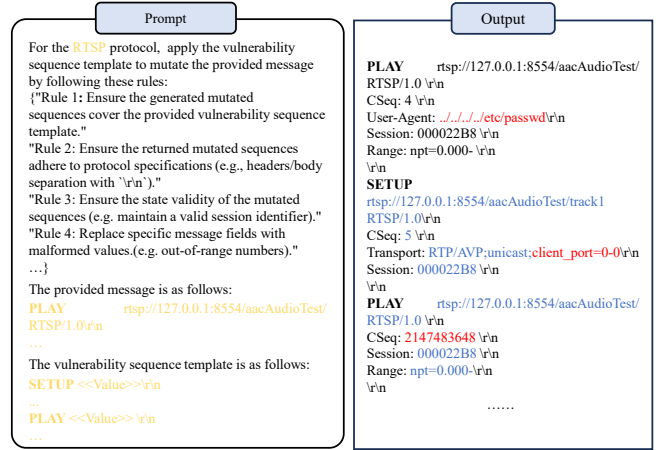


Figure 5. Prompt template used for seed mutation

3.3.2 Schedule Strategy

In directed mutation, which is based on vulnerability sequences, it is essential to select appropriate sequences to guide seed mutation, as different sequences yield different results. To address this issue, we model the sequence scheduling as a multi-armed bandit problem [36], defined as follows:

Definition 1. The multi-armed bandit problem is represented as a tuple (A, R) , where $A = \{a_1, a_2, \dots, a_k\}$ denotes a set of actions. R represents the reward probability distribution, with the reward distribution for each action defined as $R(a)$. As time progresses, the reward probability distribution for each action may change. The reward distribution for action a at time step t is denoted as $R_t(a)$. The objective of the multi-armed bandit is to maximize the accumulated reward $R_T = \sum_{t=1}^T r_t$, within a finite number of decision rounds T , where r_t is the reward obtained from the action at time t , taking values of 0 or 1.

For GLMFuzz, each vulnerability sequence can be considered as an “arm” with an unknown success rate. At any given time t , when we use the selected sequences to guide seed mutation, the test results, which are determined by code coverage and state transitions, indicate the success or failure of the action. Therefore, we employ a beta distribution to model the reward probability distribution of each sequence. Specifically, if an action has been executed $m_1 + m_2$ times, with m_1 successes and m_2 failures, the reward distribution for that action follows a beta distribution with parameters $(m_1 + 1, m_2 + 1)$. During the sequence scheduling process, we use Thompson sampling [37] to perform one round of sampling on each sequence’s probability distribution to obtain a set of reward samples. Ultimately, we select the sequences with the highest reward.

3.4 Automatic Prompt Engineering

When test cases generated by the LLM fail to improve code coverage, continued reliance on static prompt templates leads to the generation of many ineffective test cases, significantly degrading the fuzzing efficiency. To address this issue, GLMFuzz employs an automatic prompt engineering algorithm that dynamically adjusts the prompt template based on fuzzing results. Algorithm 1 illustrates the workflow of the automatic prompt engineering algorithm. The inputs to the algorithm include the initial prompt template and the message for mutation. The output is the optimized prompt template.

Algorithm 1 Automatic Prompt Engineering

Input P_0 : initial prompt template; M : message for mutation

Output P^* : optimized prompt template

```

1: /*Initialize iteration count, prompt template and template
   collection.*/*
2:  $t \leftarrow 0$ 
3:  $P_t \leftarrow P_0$ 
4:  $\mathcal{H} \leftarrow \emptyset$ 
5:  $\mathcal{S} \leftarrow \{s_1, s_2, s_3\}$  // Three optimization strategies:  $s_1$  is
   heuristic mutation,  $s_2$  is template reuse, and  $s_3$  is random
   mutation.
6: while  $t < t_{\max}$  do // Below max iterations.
7:    $M' \leftarrow \text{MutateMessage}(P_t, M)$ 
8:    $R \leftarrow \text{SendToServer}(M')$ 
9:   if  $\text{IsEffective}(R)$  then
10:    /*Add the template to the template collection and
      reuse the template.*/*
11:     $\mathcal{H} \leftarrow \mathcal{H} \cup \{P_t\}$ 
12:     $P_{t+1} \leftarrow P_t$ 
13:   else
14:    /*Select a strategy for prompt update.*/*
15:     $s \leftarrow \text{SelectStrategy}(\mathcal{S})$ 
16:     $P_{t+1} \leftarrow \text{UpdatePrompt}(P_t, s, \mathcal{H})$ 
17:   end if
18:    $t \leftarrow t + 1$ 
19: end while
20: return  $P^* \leftarrow P_t$ 

```

The algorithm first initializes the iteration count, prompt tem-

plate, template collection and optimization strategies. During each iteration, GLMFuzz uses the current prompt template to guide the LLM in mutating message M , thereby generating the mutated message M' . Subsequently, GLMFuzz sends M' to the server for fuzzing and awaits feedback. Upon receiving the fuzzing results, GLMFuzz evaluates the effectiveness of the current prompt template by analyzing three key metrics: code coverage, state transitions, and crashes. If the test case triggers at least one metric, such as discovering a new code path, GLMFuzz places the current template into the template collection and reuses it for the next iteration. Conversely, if none of the metrics are met, GLMFuzz selects an optimization strategy, modifies the current template to generate a new prompt template, and proceeds to the next iteration.

GLMFuzz employs three optimization strategies to refine the structure of prompt templates: *heuristic mutation*, *template reuse*, and *random mutation*. Under the heuristic mutation, GLMFuzz incorporates more detailed mutation strategies into the prompt templates, such as integrating vulnerability sequence features or adding mutation rules like field length boundaries, regular expression patterns, and data type boundary values, into prompt templates, thereby creating an enhanced template with clear mutation guidance. For the template reuse, GLMFuzz randomly selects the validated effective template from the template collection to serve as the new prompt template. In the random mutation, GLMFuzz directs LLM to perform rewriting of the templates, generating a broader spectrum of potential prompt templates. GLMFuzz uses the generated prompt template as the base template for the subsequent iteration until the maximum number of iterations is reached, at which point the algorithm concludes.

4. EVALUATION

In this section, we conduct experiments to compare the performance of GLMFuzz with SOTA protocol fuzzers on text-based protocols, specifically answering the following questions:

- **RQ1:** Can GLMFuzz achieve higher state space coverage than the SOTA fuzzers?
- **RQ2:** Can GLMFuzz achieve higher code space coverage than the SOTA fuzzers?
- **RQ3:** What is the impact of the key components on the performance of GLMFuzz?
- **RQ4:** Can GLMFuzz discover more vulnerabilities than the SOTA fuzzers do?

Table 1. Detailed information on the target subjects.

Subject	Protocol	Size	Language	Version
Live555	RTSP	57k	C++	31284aa
Kamailio	SIP	939k	C	a220901
Forked-daapd	DAAP	79k	C	2ca10d9
ProFTPD	FTP	242k	C	61e621e
PureFTPD	FTP	29k	C	10122d9
Exim	SMTP	118k	C	d6a5a05

Table 2. Average number of state transitions for GLMFuzz and the baselines over 10 runs of 24 hours.

Subject	GLMFuzz	Comparison with AFLNET			Comparison with CHATAFL		
		AFLNET	Improv	$\hat{A}_{12}$	CHATAFL	Improv	$\hat{A}_{12}$
Live555	152.40	96.20	58.42%	1.00	143.40	6.28%	0.80
Kamailio	128.90	92.60	39.20%	1.00	106.90	20.58%	0.93
Forked-daapd	27.10	19.70	37.56%	1.00	21.40	26.64%	0.99
ProFTPD	313.00	150.80	107.56%	1.00	266.00	17.67%	0.97
PureFTPD	320.80	224.90	42.64%	1.00	274.00	17.08%	1.00
Exim	121.80	87.20	39.68%	1.00	110.10	10.63%	0.96
AVG	-	-	54.18%	-	-	16.48%	-

4.1 Experimental Setup

Implementation. GLMFuzz is implemented using the protocol fuzzing framework CHATAFL [12]. We collect 253 vulnerability sequences from CVE database or GitHub historical issues, utilize the Embedding-3 model for vectorization, and construct the sequence feature vector database. During the sequence collection stage, we do not reproduce these vulnerabilities. We just ensure that the collected vulnerability sequences do not trigger vulnerabilities in the tested protocol implementation, thereby preventing any impact on the validity of our experimental results.

During the seed initialization and mutation stage, we employ the CHATGLM-4-PLUS model as LLM and configure the LLM’s temperature parameter to 0.5 to ensure the generation of high-quality seeds that also strictly adhere to protocol syntax. Throughout our experiments, we interact with the LLM exclusively via API calls, which is highly cost-effective with usage priced at approximately ¥0.005 per 1,000 tokens.

Furthermore, within the automatic prompt engineering algorithm, our optimization process executes a maximum of 64 iterative cycles to balance prompt generation efficacy with LLM interaction costs, exploring potential templates while minimizing time consumption from frequent LLM communications during fuzzing.

Benchmark. Our experimental setup employs six text-based network protocol implementations from the PROFUZZBENCH benchmark [18], as detailed in Table 1. The selected implementations encompass mainstream implementations of RTSP, SIP, DAAP, FTP, and SMTP.

Baselines. We select AFLNET [13] and CHATAFL as baseline tools for protocol fuzzing. AFLNET pioneered a gray-box fuzzer designed for protocol implementations while generating test cases through mutation-based strategies and guiding the fuzzing process via code coverage and state feedback. CHATAFL abstracts protocol syntax through LLMs and leverages this knowledge to drive seed expansion and mutation. Additionally, in the CHATAFL tool, we have also utilized the CHATGLM-4-PLUS model to ensure the accuracy of the experiments.

Environment. The experimental environment utilized an Ubuntu 20.04.2 LTS server configured with an Intel Xeon Silver 4210R CPU (2.40GHz) and 256GB of RAM. Each test was performed in a Docker container containing both the protocol implementation and fuzzing tools, with the code coverage and

other metrics continuously recorded by automated scripts.

4.2 State Coverage

Transition. State transitions reflect the depth of exploration within finite-state machines, making them a crucial evaluation metrics in protocol fuzzing. To quantify the improvement of GLMFuzz over the baselines, we collected state transition data over 10 runs within 24 hours. Table 2 shows the average number of state transitions explored by GLMFuzz and the two baseline tools, providing the percentage improvement in state coverage in 24 hours (Improv) and the probability that GLMFuzz is superior to the baselines ($\hat{A}_{12}$, Vargha-Delaney measure of effect size [38]).

Overall, GLMFuzz achieves more state transitions. Compared to AFLNET, GLMFuzz increases state transitions by 54.18%. Specifically, in the ProFTPD, GLMFuzz had the highest degree of improvement, increasing the number of state transitions by 107.56%. Compared to CHATAFL, GLMFuzz yields an average of 16.48% more state transitions. For all protocols tested, $\hat{A}_{12} \geq 0.80$ demonstrates significant superiority of GLMFuzz in state transition coverage compared to both baseline fuzzers.

Table 3. Average number of states covered by fuzzers.

Subject	GLMFuzz	AFLNET	Improv	CHATAFL	Improv
Live555	14.40	12.10	19.01%	14.00	2.86%
Kamailio	15.10	13.70	10.22%	13.70	10.22%
Forked-daapd	8.80	8.10	8.64%	8.40	4.76%
ProFTPD	28.90	21.10	36.97%	27.40	5.47%
PureFTPD	29.60	27.40	8.03%	28.70	3.14%
Exim	18.40	17.60	4.55%	17.80	3.37%
AVG	-	-	14.57%	-	4.97%

State. Table 3 shows the average number of states covered by GLMFuzz and the two baseline tools, providing the percentage improvement in state coverage within 24 hours (Improv). In general, GLMFuzz is able to explore more states than both baselines, with increases of 14.57% over AFLNET and 4.97% over CHATAFL. Specifically, in the case of Kamailio, GLMFuzz explores an average of 15.1 states, whereas AFLNET and CHATAFL cover only 13.4 and 13.7 states, respectively. Only in the Forked-daapd experiment, does GLMFuzz achieve average states similar to those of AFLNET and CHATAFL.

In summary, in terms of state coverage, GLMFuzz covers an average of 54.18% and 16.48% more state transitions than AFLNET and CHATAFL, respectively. In addition, GLMFuzz

Table 4. Average number of branches covered by GLMFuzz and the baselines in 10 runs of 24 hours.

Subject	GLMFuzz	Comparison with AFLNET			Comparison with CHATAFL		
		AFLNET	Improv	$\hat{A}_{12}$	CHATAFL	Improv	$\hat{A}_{12}$
Live555	2984.10	2868.70	4.02%	1.00	2891.00	3.22%	1.00
Kamailio	10500.20	9444.50	11.18%	1.00	9803.40	7.11%	1.00
Forked-daapd	2394.00	2268.00	5.56%	1.00	2269.10	5.50%	0.97
ProFTPD	5543.70	4814.10	15.16%	1.00	5160.80	7.42%	1.00
PureFTPD	1203.50	1014.80	18.59%	1.00	1084.20	11.00%	1.00
Exim	3995.30	3661.10	9.13%	1.00	3706.70	7.79%	1.00
AVG	-	-	10.61%	-	-	7.01%	-

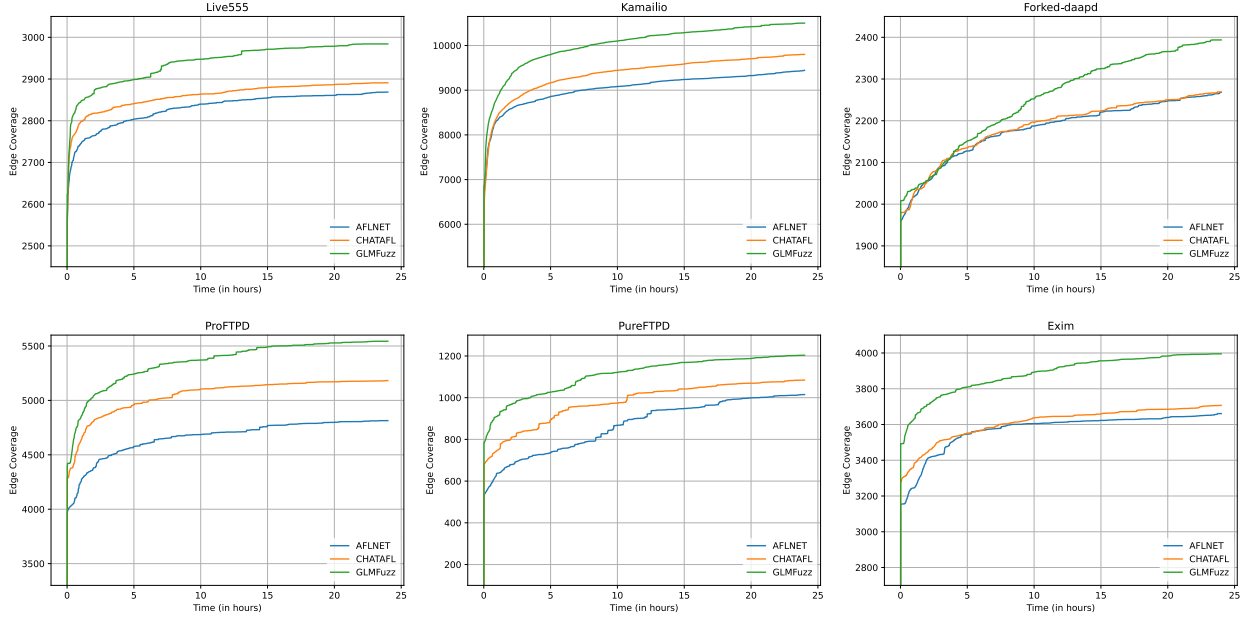


Figure 6. Edge coverage of different fuzzers running for 24 hours on the six protocol implementations.

also explores a reachable state space than both baseline fuzzers do.

4.3 Code Coverage

Code coverage quantifies fuzzer effectiveness through measuring the number of code execution paths covered during testing. Through 10 fuzzing campaigns in 24 hours, Table 4 shows the average number of branches covered by GLMFuzz and the baselines, providing the percentage improvement in code coverage in 24 hours (Improv) and the probability that GLMFuzz is superior to the baselines ($\hat{A}_{12}$).

The results show that in all six experiments, GLMFuzz covers more branches than both baselines do. Specifically, compared with AFLNET, GLMFuzz improves the code coverage by 10.61%. In experiments with Kamailio, ProFTPD, and PureFTPD, GLMFuzz achieves code coverage improvements exceeding 10%, reaching 11.18%, 15.16%, and 18.59% respectively. Compared with CHATAFL, GLMFuzz improves the code coverage by 7.01%. In experiments with Kamailio, Forked-daapd, Proftpd, PureFTPD and Exim, GLMFuzz

achieves code coverage improvements exceeding 5%, with the most significant enhancement observed in the PureFTPD experiment, reaching 11.00%. For all protocol implementations, $\hat{A}_{12} \geq 0.97$ indicates that GLMFuzz has a substantial advantage over the two baselines in terms of code coverage. Additionally, Figure 6 shows the edge coverage of different fuzzers running for 24 hours on the six protocol implementations. In all six experiments, GLMFuzz improves the code coverage more efficiently than AFLNET and CHATAFL do. However, in the Forked-daapd experiment, the results of the three tools are very similar for a brief period.

In summary, in terms of code coverage, GLMFuzz covers an average of 10.61% and 7.01% more branches than AFLNET and CHATAFL and also has the fastest exploration speed.

4.4 Ablation Study

To investigate the contribution of each component of GLMFuzz to the overall fuzzing effectiveness, we conducted ablation studies based on two key components of GLMFuzz: (a) basic fuzzing, including initial seed expansion, mutation

Table 5. Average number of branches covered by CHATAFL, CL0 (CHATAFL with initial seed expansion strategy), CL1 (CHATAFL with initial seed expansion and mutation strategy), and CL2 (CHATAFL with all strategies)

Subject	CHATAFL	Comparison with CL0			Comparison with CL1			Comparison with CL2		
		CL0	Improv	$\hat{A}_{12}$	CL1	Improv	$\hat{A}_{12}$	CL2	Improv	$\hat{A}_{12}$
Live555	2891.00	2913.20	0.77%	0.91	2930.70	1.37%	0.98	2984.10	3.22%	1.00
Kamailio	9803.40	9927.80	1.27%	0.80	10225.80	4.31%	0.95	10500.20	7.11%	1.00
Forked-daapd	2269.10	2310.30	1.82%	0.72	2334.80	2.90%	0.78	2394.00	5.50%	0.97
ProFTPD	5160.80	5290.80	2.52%	0.90	5436.90	5.35%	1.00	5543.70	7.42%	1.00
PureFTPD	1084.20	1089.40	0.48%	0.65	1138.10	4.97%	0.83	1203.50	11.00%	1.00
Exim	3706.70	3894.30	5.06%	0.96	3931.30	6.06%	1.00	3995.30	7.79%	1.00
AVG	-	-	1.99%	-	-	4.16%	-	-	7.01%	-

based on vulnerability knowledge, and a scheduling strategy;
(b) automatic prompt engineering.

4.4.1 Basic fuzzing

We conducted a systematic evaluation of three core strategies within the base fuzzing: initial seed expansion, mutation based on vulnerability sequences, and a scheduling strategy. Through an ablation study, we analyzed four configurations: CHATAFL (baseline without any strategy), GLMFuzz-CL0 (CHATAFL with initial seed expansion strategy), GLMFuzz-CL1 (CHATAFL with initial seed expansion and mutation strategy) and GLMFuzz-CL2 (CHATAFL with all strategies). As with previous experiments, we collected code coverage data over 10 runs within 24 hours. Table 5 presents the coverage metrics, including coverage improvement percentages (Improv) over the baseline and the probability of outperforming the baselines ($\hat{A}_{12}$).

Overall, all the strategies contribute to improve code coverage, with no strategy having a negative impact on coverage. Specifically, compared with CHATAFL, GLMFuzz-CL0, GLMFuzz-CL1, and GLMFuzz-CL2 increase the average code coverage by 1.99%, 4.16%, and 7.01%, respectively.

For CL0, we evaluate the effectiveness of seed expansion based on the LLM. In four experiments, Proftpd, Forked-daapd, Exim, and Kamailio, the code coverage increased by 2.52% to 1.27% respectively. Only PureFTPD and Live555 show improvements of less than 1%, at 0.48% and 0.77%, respectively.

For CL1, compared with CL0, CL1 improves the average code coverage by 2.17%, reaching 4.16%. The average code coverage improvements for Kamailio, ProFTPD, PureFTPD and Exim, exceeded 4%, ranging from 4.31% to 6.06%. Notably, in the PureFTPD, mutation based on the vulnerability sequence strategy improves code coverage by an additional 4.49%, reaching 4.97%. For the live555, the improvement is only 0.6%, reaching 1.37%, with limited improvement.

For CL2, compared with CL1, for all the experimental results, CL2 increases the average code coverage by 1.73% to 6.03%, ultimately improving the code coverage by 7.01% compared to the baseline. Specifically in ProFTPD, the scheduling strategy helps increase edge coverage by 6.03%.

4.4.2 Automatic prompt engineering

We conduct experiments to evaluate the effectiveness of the automatic prompt engineering (APE) algorithm by testing GLMFuzz with and without APE. Two tools are compared: GLMFuzz (with model optimization) and GLMFuzz-NoAPE (without APE). We collect data on average state transitions and code coverage from 10 rounds of 24-hour fuzzing for both tools. Table 6 presents the average coverage metrics from this ablation study.

Table 6. Average number of state transitions and branches covered by GLMFuzz-NoAPE and GLMFuzz.

Subject	GLMFuzz-NoAPE		GLMFuzz			
	Trans	Cov	Trans	Improv	Cov	Improv
Live555	150.80	2936.70	152.40	1.06%	2984.10	1.61%
Kamailio	121.10	10316.00	128.90	6.44%	10500.20	1.79%
Forked-daapd	25.30	2344.70	27.10	7.11%	2394.00	2.10%
ProFTPD	290.80	5467.20	313.00	7.63%	5543.70	1.40%
PureFTPD	309.10	1150.00	320.80	3.79%	1203.50	4.65%
Exim	119.30	3953.90	121.80	2.10%	3995.30	1.05%
AVG	-	-	-	4.69%	-	2.10%

The results demonstrate that GLMFuzz outperforms GLMFuzz-NoAPE across the six protocol experiments in both state transitions and code coverage. Specifically, for state transitions, GLMFuzz shows improvements ranging from 1.06% to 7.63%, with an average enhancement of 4.69%. The most significant improvements are observed in Forked-daapd and ProFTPD, which are 7.11% and 7.63%, respectively. With respect to code coverage, GLMFuzz achieved improvements between 1.05% and 4.65%, averaging 2.10% across all the experiments. In particular, PureFTPD achieves a maximum code coverage improvement of 4.65% through APE.

In conclusion, the integration of automated prompt engineering algorithm in GLMFuzz significantly enhances both state transitions and code coverage across all protocol tests compared to its GLMFuzz-NoAPE. These improvements validate the effectiveness of APE in optimizing LLM-based fuzzing performance.

4.5 Vulnerability Discovery

Vulnerability discovery serves as a critical metric for evaluating the performance of the fuzzer. To assess GLMFuzz’s effectiveness in crash detection, we conduct 10 independent 24-hour fuzzing campaigns, recording the number of unique

crashes triggered during each experiment, as presented in Table 7.

Table 7. Number of unique crashes found by different fuzzers.

Subject	GLMFuzz	AFLNET	CHATAFL
Live555	24	3	1
Kamailio	97	65	82
Forked-daapd	2	0	0
ProFTPD	49	44	47
PureFTPD	0	0	0
Exim	0	0	0

The experimental results show that in four out of the six experiments, GLMFuzz discovers more crashes than the other two baseline tools do. Specifically, in the Live555 experiment, GLMFuzz found 24 crashes, significantly surpassing AFLNET (3 crashes) and CHATAFL (1 crash). In the Forked-daapd experiment, GLMFuzz detected 2 crashes, whereas neither AFLNET nor CHATAFL identified any vulnerabilities. However, no crashes were detected by any fuzzer in the PureFTPD and Exim protocol implementations.

Furthermore, through systematic analysis of crash cases triggered by various fuzzing tools, GLMFuzz identified a previously unknown vulnerability in the Forked-daapd experiment. This is a stack overflow vulnerability caused by infinite recursion when the DAAP protocol packet parameters contain a large number of left parentheses. We have submitted a vulnerability report to the developers.

In summary, compared with the baseline fuzzers AFLNET and CHATAFL, GLMFuzz demonstrates superior ability to discover vulnerabilities, resulting in enhanced effectiveness in identifying crashes across diverse protocol implementations. Moreover, GLMFuzz also finds a previously unknown vulnerability in Forked-daapd, which has been assigned CVE identifier.

5. RELATED WORK

Protocol Fuzzing. The protocol fuzzing is divided into generation-based fuzzing and mutation-based fuzzing. Generation-based fuzzing, such as Peach [23], Snooze [24], and Peach* [39], provides standard interfaces via configuration files. This approach requires users to manually model protocol syntax, which is a time-consuming and error-prone process. Additionally, these tools demonstrate limited capability in effectively exploring the protocol state space. Therefore, mutation-based protocol fuzzing has become a key research focus. Fuzzing tools such as AFLNET[13], CHATAFL [12], and NSFUZZ [15] interact with server by sending mutated protocol messages and extracting feedback to guide mutation. In this paper, we leverage mutation-based protocol fuzzing to test target systems by utilizing LLMs combined with vulnerability sequence knowledge for generating and mutating protocol messages.

Fuzzing with Historical Bug. Researchers have long explored leveraging historical vulnerability data to enhance fuzzing techniques. LangFuzz [40] parses syntax to generate code

components and learns error-triggering code snippets from historical tests, integrating them into mutated test cases. JavaTailor [16] analyzes test programs from historical JVM vulnerabilities, extracts critical code snippets, and generates diverse test programs to achieve comprehensive coverage of JVM behaviors. FuzzGPT [41] fine-tunes LLM using historical vulnerability data from deep learning APIs, automatically generates test programs, and employs heuristic mutation algorithms to strengthen its fuzzing capabilities for deep learning APIs. In this work, we use historical vulnerability sequences to improve the quality of initial seeds and guide LLM in performing directed mutation to seeds.

Fuzzing Based on Large Language Models. Large Language Models (LLMs) possess robust natural language processing capabilities, and security researchers have integrated them with fuzzing techniques to explore their effectiveness in seed generation and mutation. TitanFuzz [31] employs generative LLM to produce large volumes of deep learning API code and tests them with diverse mutation operations during fuzzing. CHATAFL [12] extracts the grammar of network protocols through LLM and generates messages to expand the initial seed corpus. InputBlaster [42] combines generative LLM with UIAutomator4 to automatically generate malformed text inputs to detect mobile application vulnerabilities. In this work, GLMFuzz uses historical vulnerability sequences to perform directed mutations on message sequences, and employs LLM to generate test cases that comply with protocol specifications.

Prompt Engineering in Fuzzing. Integrating LLM into fuzzing requires effective prompt engineering. Fuzz4All [32] pioneered the fusion of prompt engineering with seed mutation strategies. The system starts with arbitrary user inputs as initial prompts and iteratively refines them during the fuzzing loop. ParaFuzz [43] highlights the challenge of identifying synonyms and intersections between content words and function words during mutation. To preserve semantic integrity during code transformation, they propose synonym substitution via a meta-prompt framework powered by ChatGPT. In this work, we introduce an automatic prompt engineering algorithm that dynamically adjusts prompt templates via three optimization strategies by evaluating fuzzing results.

6. CONCLUSION

In this paper, we propose GLMFuzz, which is a vulnerability-knowledge-guided fuzzing method for protocol implementations based on large language models(LLMs). Historical vulnerability sequences frequently contain special states and field information that can trigger boundary conditions and exceptional behaviors in protocol implementations, which can enhance the ability of LLM to generate high-quality test cases. Therefore, GLMFuzz constructs a vulnerability knowledge vector database based on these sequences and uses LLM to generate diverse initial seeds. During the fuzzing process, GLMFuzz employs a directed mutation based on vulnerability sequences with the highest reward. In addition, GLMFuzz employs an automatic prompt engineering algorithm with three optimization strategies to dynamically adjust prompt templates

based on fuzzing results. To validate the performance of GLMFuzz, we conducted a comparative evaluation against two SOTA tools on six real-world protocol implementations. The evaluation results confirm that GLMFuzz outperforms the baseline tools in terms of exploring state transitions, discovering new states and improving code coverage. Moreover, GLMFuzz also identifies a previously unknown vulnerability in Forked-daapd, which has been assigned CVE identifier.

ACKNOWLEDGMENT

The work is supported by the Major Key Project of PCL (Grant No. PCL2024A05) and the National Natural Science Foundation of China (Grant No. 62172191).

REFERENCES

- [1] H. Huni, R. Johnson, and R. Engel, "A framework for network protocol software," *Sigplan Notices*, vol. 30, no. 10, pp. 358-369, Oct, 1995.
- [2] Z. Luo, J. Yu, F. Zuo, J. Liu, Y. Jiang, T. Chen, A. Roychoudhury, and J. Sun, "Bleem: Packet sequence oriented fuzzing for protocol implementations," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4481-4498.
- [3] Z. Durumeric, F. Li, J. Kasten, J. Amann, J. Beekman, M. Payer, N. Weaver, D. Adrian, V. Paxson, and M. Bailey, "The matter of heartbleed," in *Proceedings of the 2014 conference on internet measurement conference*, 2014, pp. 475-488.
- [4] S. Mohurle and M. Patil, "A brief study of wannacry threat: Ransomware attack 2017," *International journal of advanced research in computer science*, vol. 8, no. 5, pp. 1938-1940, 2017.
- [5] B. Hawkes, "0day In the Wild," 2019. Available online: <https://googleprojectzero.blogspot.com/p/0day.html>.
- [6] H. O'Neill, "2021 has broken the record for zero-day hacking attacks," 2021. Available online: <https://www.technologyreview.com/2021/09/23/1036140/2021-record-zero-day-hacks-reasons/>.
- [7] P. Oehlert, "Violating assumptions with fuzzing," *Ieee Security & Privacy*, vol. 3, no. 2, pp. 58-62, Mar-Apr, 2005.
- [8] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A Survey for Roadmap," *Acm Computing Surveys*, vol. 54, no. 11S, Jan, 2022.
- [9] C. Daniele, S. B. Andarzian, and E. Poll, "Fuzzers for Stateful Systems: Survey and Research Directions," *Acm Computing Surveys*, vol. 56, no. 9, Sep, 2024.
- [10] C. Chen, B. Cui, J. Ma, R. Wu, J. Guo, and W. Liu, "A systematic review of fuzzing techniques," *Computers & Security*, vol. 75, pp. 118-137, Jun, 2018.
- [11] H. Wei, Z. Du, H. Huang, Y. Liu, G. Cheng, L. Wang, and B. Mao, "Inferring State Machine from the Protocol Implementation via Large Language Model," *arXiv preprint arXiv:2405.00393*, 2024.
- [12] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024, vol. 2024.
- [13] V.-T. Pham, M. Böhme, and A. Roychoudhury, "Aflnet: A greybox fuzzer for network protocols," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020, pp. 460-465.
- [14] R. Natella, "StateAFL: Greybox fuzzing for stateful network servers," *Empirical Software Engineering*, vol. 27, no. 7, Dec, 2022.
- [15] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, and C. Zhang, "NSFuzz: Towards Efficient and State-Aware Network Service Fuzzing," *Acm Transactions on Software Engineering and Methodology*, vol. 32, no. 6, Nov, 2023.
- [16] Y. Zhao, Z. Wang, J. Chen, M. Liu, M. Wu, Y. Zhang, and L. Zhang, "History-driven test program synthesis for JVM testing," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1133-1144.
- [17] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, vol. 2, 2023.
- [18] R. Natella and V.-T. Pham, "Profuzzbench: A benchmark for stateful protocol fuzzing," in *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, 2021, pp. 662-665.
- [19] J. Shi, Z. Wang, Z. Feng, Y. Lan, S. Qin, W. You, W. Zou, M. Payer, and C. Zhang, "AIFORE: Smart Fuzzing Based on Automatic Input Format Reverse Engineering," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4967-4984.
- [20] A. Bulekov, B. Das, S. Hajnoczi, and M. Egele, "No grammar, no problem: Towards fuzzing the linux kernel without system-call descriptions," in *Network and Distributed System Security (NDSS) Symposium*, 2023.
- [21] Q. Liu, F. Toffalini, Y. Zhou, and M. Payer, "Videzzo: Dependency-aware virtual device fuzzing," in *2023 IEEE Symposium on security and privacy (SP)*, 2023: IEEE, pp. 3228-3245.
- [22] M. Cheng, K. Zhu, Y. Chen, G. Yang, Y. Lu, and C. M. Lu, "Augmenting Protocol Fuzzing with Message Syntax Comprehension via Large Language Models," *Electronics*, vol. 13, pp. 2632, 2024.
- [23] M. EddingtonEddington, Peach Fuzzing Platform. 2004, Available online: <https://gitlab.com/peachtech/peach-fuzzer-community>
- [24] G. Banks, M. Cova, V. Felmetzger, K. Almeroth, R. Kemmerer, and G. Vigna, "SNOOZE: toward a Stateful Network protocol fuzzer," in *Information Security: 9th International Conference, ISC 2006, Samos Island, Greece, August 30-September 2, 2006. Proceedings 9*, 2006: Springer, pp. 343-358.
- [25] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, and

- A. Askill, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877-1901, 2020.
- [26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [27] R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, and Y. Du, "Lamda: Language models for dialog applications," *arXiv preprint arXiv:2201.08239*, 2022.
- [28] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, and S. Anadkat, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [29] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, "Evaluating large language models in class-level code generation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1-13.
- [30] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1865-1879.
- [31] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, 2023, pp. 423-435.
- [32] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1-13.
- [33] J. Hu, Q. Zhang, and H. Yin, "Augmenting grey-box fuzzing with generative ai," *arXiv preprint arXiv:2306.06782*, 2023.
- [34] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM computing surveys*, vol. 55, no. 12, pp. 1-38, 2023.
- [35] S. Hambardzumyan, A. Tuli, L. Ghukasyan, F. Rahman, H. Topchyan, D. Isayan, M. McQuade, M. Harutyunyan, T. Hakobyan, and I. Stranic, "Deep lake: A lakehouse for deep learning," *arXiv preprint arXiv:2209.10785*, 2022.
- [36] S. Bubeck, and N. Cesa-Bianchi, "Regret analysis of stochastic and nonstochastic multi-armed bandit problems," *Foundations and Trends® in Machine Learning*, vol. 5, no. 1, pp. 1-122, 2012.
- [37] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on thompson sampling," *Foundations and Trends® in Machine Learning*, vol. 11, no. 1, pp. 1-96, 2018.
- [38] A. Arcuri, and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219-250, 2014.
- [39] Z. Luo, F. Zuo, Y. Shen, X. Jiao, W. Chang, and Y. Jiang, "ICS protocol fuzzing: Coverage guided packet crack and generation," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*, 2020: IEEE, pp. 1-6.
- [40] C. Holler, K. Herzig, and A. Zeller, "Fuzzing with code fragments," in *21st USENIX Security Symposium (USENIX Security 12)*, 2012, pp. 445-458.
- [41] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt," *arXiv preprint arXiv:2304.02014*, 2023.
- [42] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, Z. Tian, Y. Huang, J. Hu, and Q. Wang, "Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model," in *Proceedings of the IEEE/ACM 46th international conference on software engineering*, 2024, pp. 1-12.
- [43] L. Yan, Z. Zhang, G. Tao, K. Zhang, X. Chen, G. Shen, and X. Zhang, "Parafuzz: An interpretability-driven technique for detecting poisoned samples in nlp," *Advances in Neural Information Processing Systems*, vol. 36, pp. 66755-66767, 2023.